

IAI group codes standards

Introduction

The main purpose of this standard is to standardise everyone's code so that it can be easily read and modified by others, it also helps with collaboration and debugging, and it is suitable for internal use within the IAI team and everyone should use it when writing code. Everyone needs to deliver the code after completing the project, and someone else needs to read the code to make sure that he can read it completely. If you have questions about python, refer to the book Python Crash Course by Eric Matthes.

Contents

Introduction.....	1
1. Basic rules.....	3
1.1. Naming rules	3
1.1.1. Variables, functions and files.....	3
1.1.2. Classes.....	3
1.1.3. Constants.....	3
1.2. Indentations and spaces	4
1.2.1. Indentations.....	4
1.2.2. Length	4
1.3. Comments.....	4
1.3.1. Line comments.....	4
1.3.2. Function, class and module.....	4
1.3.3. Import module.....	5
1.3.4. File	5
2. Code structure.....	6
2.1. Code Introduction.....	7
2.1.1. Main demo file.....	7
2.1.2. Function library.....	8
2.1.3. Basis function.....	9
2.2. Roslaunch file.....	12

1. Basic rules

1.1. Naming rules

1.1.1. Variables, functions and files

Variables, functions and files should be descriptive, avoiding abbreviations or overly complex names. The common naming convention is to use **lowercase letters with underscores**.

Example:

☑Correct:

```
user_name = "Tony"
get_user_info()
```

☒Incorrect:

```
userName = "Tony"
a = "Tony"
GetUserInfo()
```

1.1.2. Classes

The first letter of each word is **capitalized** and there is **no underlining** between words.

Example:

☑Correct:

```
class UserProfile:
```

☒Incorrect:

```
class user_profile:
```

1.1.3. Constants

Constants should be descriptive, avoiding abbreviations or overly complex names. They are all **capital letters, with words separated by underscores**.

Example:

☑Correct:

```
MAX_CONNECTIONS = 100
PI = 3.14159
```

☒Incorrect:

```
max_connections = 100
pi = 3.14159
```

1.2. Indentations and spaces

1.2.1. Indentations

Use 4 spaces for indentation instead of tabs.

Example:

☒ **Correct:**

```
def my_function():
    print("Hello, world!")
```

1.2.2. Length

Each line of code should **not exceed 80** characters. If a line is too long, you should use **brackets or backslashes (\) for line breaks**.

Example:

☒ **Correct:**

```
long_variable_name = (
    " Each line of code should not exceed 80 characters. If a line is too long,"
    " you should use brackets or backslashes (\) for line breaks."
)
```

1.3. Comments

1.3.1. Line comments

Use the # symbol for line comments, which should be located on the code or after the code that needs to be explained. Try to have a comment on every line to make sure that everyone else can read it.

Example:

☒ **Correct:**

```
# This is algorithm setting
max_connections = 100 # This is the number of algorithmic loops
pi = 3.14159          # This is pi  $\pi$ 
```

1.3.2. Function, class and module

Use documentation **strings (docstring)** for functions, classes and modules

Example:

☒ **Correct:**

```
def my_function():
    """
    This function is used for robot gripper control, there are two models to control it, open gripper
    and close gripper.
    """
    print("Hello, world!")
```

1.3.3. Import module

Place all imported modules at the **top of the codes** in the order of standard, third-party, and local libraries, sorted alphabetically within each group.

Example:

☑Correct:

```
# Standard library, this is use for basic system functions.
import os
import sys

# Third-party library module, this is use for drawing. You should download numpy first.
import numpy as np

# Local library module, this is use for robot gripper control.
from my_module import my_function
```

1.3.4. File

Use `"""` to give the comments.

Example:

☑Correct:

```
"""
Example usage:
from function_library.voice import VoiceAssistant
voice.text_to_speech("Hello, how can I assist you?")
recognized_text = voice.transcript_queue.get(timeout=5)
print(f"Main function received: {recognized_text}")
if "who are you" in recognized_text.lower():
    voice.text_to_speech("I am Tina.")
"""
```

2. Code structure

The code is structured according to the following, all filenames are lowercase and words are separated by underscores.:

hrc_352_lab/

```
└── src/
    ├── driver/      #All of the above are ROS driver packages, don't change them!
    │   ├── azure_kinect_ros_driver/
    │   ├── fmauch_universal_robot/
    │   ├── gripper_breakdown_solution/
    │   ├── robotiq_ft_sensor/
    │   ├── vention_conveyor/
    │   ├── universal_robots_ros_driver/
    │   ├── robotiq_85_gripper/
    │   └── yolov5_ros/
    ├── lab_codes/
    │   ├── CMakeLists.txt
    │   ├── package.xml
    │   ├── setup.py      #It is used to declare Python packages
    │   └── launch/      #It is used to launch all drivers
    │       ├── run_gripper_transform_rviz.launch
    │       └── run_ur_azure.launch
    └── scripts/
        ├── main_demo_voice_pick_Tony.py #Create a new python file based on
        this demo and name the function and author name.
        ├── main_demo_****.py
        ├── main_demo_****.py
        └── function_library/
            ├── __init__.py
            ├── collision_avoidance.py
            ├── voice.py
            ├── gesture.py
            └── basic_function/
                ├── __init__.py
                └── body_track.py
```

```

|—— conveyor_control.py
|—— gripper_control.py
|—— open_gripper_ft300_sensor.py
|—— robot_move.py
|—— tool_detect_pick_camera.py

```

2.1. Code Introduction

2.1.1. Main demo file

(1) As you can see, you can use the library functions in this main function file, which need to be imported and initialized before using them. You should create a new python file based on the example demo and name the function and author name, such as “main_demo_voice_pick_Tony.py”. **You can change them!!!**

Example:

☑Correct:

```

from function_library.basis_function.robot_move import UR5eRobot
from function_library.voice import VoiceAssistant

```

```

if __name__ == "__main__":
    rospy.init_node("voice_pick")
    robot = UR5eRobot()
    gripper = Robotiq85Gripper()
    ft300_handler = FT300SensorHandler()
    credentials_path = "/home/zzq/psyched-thunder-454702-f7-a5bf7cfac33f.json" #
    Path to your Google Cloud API credentials
    voice = VoiceAssistant(credentials_path)
    # Initialize the gripper
    gripper = Robotiq85Gripper()
    # Initialize the FT300 sensor handler
    ft300_handler = FT300SensorHandler()

```

(2) You should use threads or subscription topics for separate loops of execution other than the main function.

Example:

☑Correct:

```
# thread
recognition_thread = threading.Thread(target = voice.speech_to_text_recognize)
recognition_thread.daemon = True
recognition_thread.start()

# subscription topic
sensor_control_thread = rospy.Subscriber('/robotiq_ft_wrench', WrenchStamped,
ft300_handler.ft300s_callback)
```

2.1.2. Function library

There are three files, including voice, gesture and collision_avoidance. They are used for main demo files. **Don't change them!!!**

Follow the tips below to use them:

(1) Voice:

It is used to speech to text and text to speech. You should get your own API credential in Google Cloud and add the right path when you use it.

Don't change them!!!

"""

Example usage:

```
from function_library.voice import VoiceAssistant
credentials_path = "/home/352lab/hrc_real/hrc_ws/hrc_interface/scripts/psyched-
thunder-454702-f7-a5bf7cf33f.json" # Path to your Google Cloud API credential
voice = VoiceAssistant(credentials_path)
```

Example: Convert text to speech

```
voice.text_to_speech("Hello, how can I assist you?")
```

Example: Start speech-to-text recognition

Create a thread to run speech_to_text_recognize()

```
recognition_thread = threading.Thread(target=assistant.speech_to_text_recognize)
recognition_thread.daemon = True
recognition_thread.start()
```

```
recognized_text = voice.transcript_queue.get(timeout=5)
```



```

print(f'Main function received: {recognized_text}')
if "who are you" in recognized_text.lower():
    voice.text_to_speech("I am Tina.")
"""

```

(2) Gesture

(3) Collision avoidance

2.1.3. Basis function

There are six files, including conveyor control, gripper control, open gripper ft300 sensor, robot move, tool detect pick camera, and body track. They are used for main demo files and function libraries. **Don't change them!!!**

Follow the tips below to use them:

(1) Conveyor control

It is used to control the conveyor belt.

Don't change them!!!

"""

Example usage:

```

from function_library.basis_function.conveyor_control import ConveyorBelt
# Initialize the conveyor belt
conveyor = ConveyorBelt()
# Move the conveyor to a specific position with speed and acceleration
conveyor.conveyor_move(speed=100.0, acceleration=100.0, position=270.0)
conveyor.go_home()
"""

```

(2) Gripper control

It is used to control the gripper. You can close the gripper with specific force.

Don't change them!!!

"""

Example usage:

```

from function_library.basis_function.gripper_control import Robotiq85Gripper

```

```

gripper = Robotiq85Gripper()
# Open the gripper
gripper.open()
# Close the gripper with the default force 100.0 N
gripper.close()
# Close the gripper with a specific force 150.0 N, Max force is 235.0 N
gripper.close_with_force(150.0)
# Check if the gripper is open
gripper.gripper_is_open()
# Check if the gripper is closed
gripper.gripper_is_closed()
"""

```

(3) Open gripper ft300 sensor

It is used to read the data from the ft300 sensor and will open the gripper when enough force is detected.

Don't change them!!!

"""

Example usage:

```

from function_library.basis_function.open_gripper_ft300_sensor import
FT300SensorHandler

from geometry_msgs.msg import WrenchStamped
# Initialize the gripper
gripper = Robotiq85Gripper()
# Initialize the FT300 sensor handler
ft300_handler = FT300SensorHandler()
sensor_control_thread = rospy.Subscriber('/robotiq_ft_wrench', WrenchStamped,
self.ft300s_callback)
"""

```

(4) Robot move

It is used to control the robot. You can move the robot to a specific position with a specific speed and acceleration. You can also get the current position of the robot.

Don't change them!!!

"""

Example usage:

```
from function_library.basis_function.robot_move import UR5eRobot
robot = UR5eRobot()
# Move to a specific position
list = []
list.append([0.7, 0.4, 0.3335, 2.231, -2.216, 0.0])
robot.execute_cartesian_trajectory(list)
# Get the current joint position or find it on the UR5e robot controller terminal interface
robot.get_actual_pose()
print("Current Cartesian Pose:", actual_pose)
# Move to home position
robot.go_home()
"""
```

(5) Tool detect pick camera

It is used to detect the tool and pick it. You should train your model by YOLOv8 and add the right path when you use it.

Don't change them!!!

"""

Usage:

```
from function_library.basis_function.tool_detect_pick_camera import
AzureKinectCamera
# Initialize the camera with the YOLOv8 model path
model_path = "path/to/your/yolov8_model.pt"
camera = AzureKinectCamera(model_path)
# Detect the tool with ID 1
camera.robot_pick_top_camera(height, tool_id)

final_coordinates = camera.robot_pick_top_camera(height, tool_id)
rospy.loginfo(f"Final picking coordinates: {final_coordinates}")
"""
```

(6) Body track

2.2. Roslaunch file

There are two launch file to launch all devices, including `run_UR_azure.launch` and `run_gripper_transform_rviz.launch`. You should launch `run_UR_azure.launch` first and then launch `run_gripper_transform_rviz.launch`.

Issues:

Having trouble with controlling the gripper# change permission for `/tmp/ttyUR`

```
sudo chmod -t /tmp
```

```
sudo chmod 777 /tmp/ttyUR
```

```
sudo chmod +t /tmp
```

Having trouble with controlling the Force/Torque sensor

change permission for `/dev/ttyUSB0`

```
sudo chmod 777 /dev/ttyUSB0
```